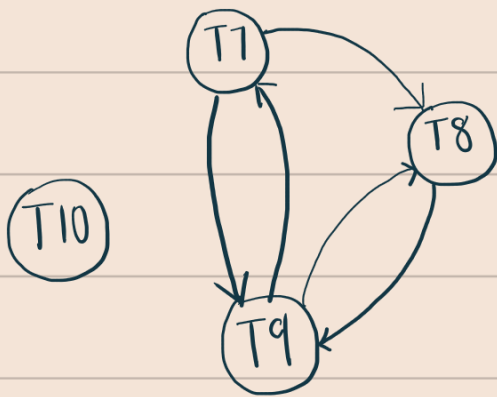


Question 3

a) i] **non-serial schedule**. The operations of transactions T7, T8, T9 and T10 are interleaved.

The transactions do not execute one after another in a strictly sequential order and overlap in time.

ii]



iii] The schedule is **not conflict-serializable**.

Conflict serializability is determined by checking whether the precedence graph contains a cycle. In this case, there is a cycle between T7 and T9 — meaning the precedence graph is cyclic.

iv] The schedule faces a **lost update** problem due to multiple transactions

(T7 and T9) update the same data

item X without proper synchronization. Updates made by one transaction can be overwritten by another, causing some updates to be lost. Also, the lack of concurrency control may also lead to inconsistent data values.

b) i]	Time	T ₁₁	T ₁₂	T ₁₃	T ₁₄
	t ₁	begin transaction			
	t ₂	lock - X (A)	begin transaction		
	t ₃	read (A)	lock - X (B)	begin transaction	
	t ₄		read(B)	lock - X(A)	begin transaction
	t ₅	A = A + 200	B = B - 50	WAIT	lock - X (C)
	t ₆	write (A)		WAIT	read (C)
	t ₇	unlock (A)	write (B)	WAIT	C = C + 10
	t ₈		unlock (B)	read (A)	write (C)
	t ₉				unlock (C)
	t ₁₀	commit		A = A - 30	commit
	t ₁₁		lock - X (A)	write (A)	
	t ₁₂		WAIT	unlock (A)	
	t ₁₃		read (A)		
	t ₁₄		A = A * 2		
	t ₁₅		write (A)		
	t ₁₆		unlock (A)		
	t ₁₇		commit	commit	
	t ₁₈				

ii] The schedule is non-serializable due to the overlap transactions. Therefore,

concurrency problem may still occur even when all transactions commit successfully

iii]	Time	T11	T12	T13	T14
	t_1	begin transaction			
	t_2	lock-X(A)	begin transaction		
	t_3	read(A)	lock-X(B)	begin transaction	
	t_4		read(B)	lock-X(A)	begin transaction
	t_5	$A = A + 200$	$B = B - 50$	WAIT	lock-X(C)
	t_6	write(A)		WAIT	read(C)
	t_7	commit/unlock(A)	write(B)	WAIT	$C = C + 10$
	t_8			read(A)	write(C)
	t_9		lock-X(A)		commit/unlock(C)
	t_{10}		WAIT	$A = A - 30$	
	t_{11}		WAIT	write(A)	
	t_{12}		WAIT	commit/unlock(A)	
	t_{13}		read(A)		
	t_{14}		$A = A * 2$		
	t_{15}		write(A)		
	t_{16}		commit/unlock(A,B)		

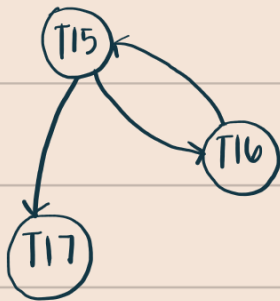
iv] **cascading rollback**, other dependent transactions such as T12 and T13 will roll back if T11 fails after unlocking A. **Starvation** may happen too because it needs to wait for the previous transaction to finish and unlock all resources.

Question 4

i] **Non-serial schedule**, because the operations of the three transactions (T_{15}, T_{16}, T_{17}) are interleaved.

For example, T_{15} begins and reads J but before it completes, T_{16} begins and operates on K and T_{17} operates on J . In a serial schedule, one transaction must commit before the next one begins.

ii]



iii] No, it is **not conflict-serializable**. There is a cycle involving T_{15} and T_{16} due to the conflicts on data item K as shown in the precedence graph. Therefore, the schedule cannot be serialized since this precedence graph is not acyclic.

iv] The schedule faces the **lost update** problem and **uncommitted dependency** (dirty read).

Lost update happens on K : T_{16} updates K at t_5 but T_{15} also updates K at t_7 based on a value read at t_5 . Since T_{15} overwrites T_{16} 's write without considering it, T_{16} 's update is lost. Meanwhile, uncommitted dependency happens on J : T_{17} reads J at t_4 immediately after T_{15} writes J at t_4 . Since T_{15} has not committed yet, T_{17} is reading dirty (uncommitted) data.

v]	Time	T_{15}	T_{16}	T_{17}
	t_1	begin transaction		
	t_2	lock-X(J)	begin transaction	

t_3	read(J)	lock-X(K)	begin transaction
t_4	$J = J + 10$	read(K)	lock-X(J)
t_5	write(J)	$K = K - 5$	WAIT
t_6	lock-X(K)	write(K)	WAIT
t_7	WAIT	commit/unlock(K)	WAIT
t_8	lock-X(K)		WAIT
t_9	read(K)		WAIT
t_{10}	$K = K + 20$		WAIT
t_{11}	write(K)		WAIT
t_{12}	commit/unlock(K, J)		WAIT
t_{13}			lock-X(J)
t_{14}			read(J)
t_{16}			$J = J - 15$
t_{17}			write(J)
t_{18}			commit/unlock(J)